Exercise Module 0.13.3 Documentation

<github.com/todo_project_repository>

Tutorial	API Reference	Works Cited

Exercise Module is a series of data analysis and graphing functions written in Python. The data analysis functions were without essential Python features such as dictionaries to familiarize the user with C-syntax limitations. The Graphing portion uses Matplotlib for RGB images.

Install	todo

Mimics built-in Python functions	CSV parser
Alphabetize with merge sort	Basic statistics
Black and white time series graph	Support the textbook at:

https://github.com/DavidLeifer/OpenSourceDocument/

Tutorials

Open a CSV and reverse the scale from 5-1 to 1-5.

```
import exercise_modules as eu

csv_path = '/path/to/file.csv'
# Names of the columns to not reverse.
unflipped_columns = ['ID', 'Date', ...etc]

# Or don't reverse column values.
unflipped_columns = 0

# Initialize the Class.
parser = eu.CSV_Parser(path)

# Function opens the file horizontally.
read = parser.file_opener()

# Index comma positions and count the columns.
comma_indexed = parser.comma_index(read, path, 0)
comma_width = parser.comma_index(read, path, 1)

# Flips the values with the number of columns.
col_width = int(((comma_width - 1) / 2 - 1)
# List to hold the values
vert = []
# Loops the width of the commas.
for i in range(0, comma_width - 1, 2):
  # Sort the 1D list into a list of columns.
  value_list = parser.csv_value_list(comma_indexed, read,
                                     col_width, i)
  # Set 'unflipped_col' to 0 to not reverse the values.
  if unflipped_col == 0:
    vert.append(value_list)
  else:
    # Ignore reversing columns if specified.
    if value_list[0] in unflipped_col:
      vert.append(value_list)
    else:
      # The function to reverse the scale.
      flip = paraser.csv_flipper(value_list, col_width)
```

```
        vert.append(flip)
print(vert)
```

Statistics between variables. `P1_vert` and `B1_vert` are two dimensional lists with each column as a list. The example skips the identification columns.

```
import exercise_modules as eu
stats_class = eu.Statistics()
# Lists to hold each column's result.
P1_means_list =  []
P1_stnd_list =  []
B1_means_list =  []
B1_stnd_list =  []
# Calculate the column's mean and standard deviation.
for i in P1_vert[3:]:
  P1_means = stats_class.mu(i)
  P1_means_list.append(P1_means)
  P1_mnt2_4 = stats_class.mnt(P1_means[0], P1_means[1], i)
  P1_stnd_list.append(P1_mnt2_4[1])
for j in B1_vert[3:]:
  B1_means = stats_class.mu(j)
  B1_means_list.append(B1_means)
  B1_mnt2_4 = stats_class.mnt(B1_means[0], B1_means[1], j)
  B1_stnd_list.append(B1_mnt2_4[1])
# Calculate covariance and correlation between B1 and P1.
for k in range(len(P1_vert[3:])):
  for m in range(len(B1_vert[2:])):
    P1B1_means = stats_class.covar(P1_means_list[k],
                                   B1_means_list[m],
                                   P1_vert[k+3],
                                   P1_vert[m+2])
    P1B1_cor = stats_class.cor(P1_means_list[k],
                               B1_means_list[m],
                               P1_vert[k+3],
                               P1_vert[m+2])
```

API Reference

Import the library, read a CSV, and get the dimensions.

```
import exercise_modules as eu
# Initialize the Class.
parser = eu.CSV_Parser(path)

# Function opens the file horizontally.
read = parser.file_opener()

# Index comma positions and count the columns.
comma_indexed = parser.comma_index(read, path, 0)
comma_width = parser.comma_index(read, path, 1)
```

exercise_modules.CSV_Parser

Example:

```
eu.CSV_Parser('/path/to/file.csv', 0)
```

Parameters:

path : ***str***

File path to the CSV or TSV files.

unflipped_columns : ***str list***

Column headers to not flip.

Returns:

The parser class object.

exercise_modules.CSV_Parser.file_opener

Returns:

Tabular file read from left to right as a one dimensional list.

exercise_modules.CSV_Parser.comma_index

Example:

```
eu.CSV_Parser(path).comma_index(read, path, 0)
eu.CSV_Parser(path).comma_index(read, path, 1)
```

Parameters:

read : ***str list***

One dimensional list of table data from `file_opener()` .

path :

comma_width : ***int***

Value of `0` or `1` .

Returns:

`0` returns an indexed list of the comma positions.

`1` returns left to right width of the columns.

exercise_modules.CSV_Parser.csv_value_list

Example:

See tutorial.

Parameters:

comma_width : ***int***

Output from `comma_indexed(read, path, 0)` .

read : ***str list***

One dimensional list of table data from `file_opener()` .

col_width : ***int***

Output from `comma_indexed(read, path, 1)` .

col_head : ***int***

Each column from `comma_indexed(read, path, 1)` (i.e. 0, 1, 2…etc).

Returns:

str list

Two dimensional list with each column as a list.

exercise_modules.CSV_Parser.csv_flipper

Example:

See tutorial.

Parameters:

csv_value_list : ***str list***

Output from `csv_value_list(comma_indexed, read, col_width, i)`.

col_width : ***int***

Output from `comma_indexed(read, path, 1)`.

Returns:

Two dimensional list with each specified column's value flipped from 5-1 to 1-5.

exercise_modules.Statistics

Example:

```
eu.Statistics()
```

Returns:

The statistics class object.

exercise_modules.Statistics.mu

Example:

```
column = ['Header', '2', '6', '3', '8', '4']
column_mean = stats_class.mu(column)
print(column_mean)

['Header', 4.6]
```

Parameters:

column : ***str list***

List of values.

Returns:

list

Header of the column as a ***str***.

The average or mean of the list as a float.

exercise_modules.Statistics.mnt

Example:

Standard deviation

$$\sigma = \sqrt{\frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}}$$

Skewness

$$\gamma_1 = \frac{1}{N} \sum_{i=1}^{N} \left(\frac{x_i - \mu}{\sigma}\right)^3$$

Kurtosis

$$s_2 = \Sigma (x - \bar{x})^2$$

$$s_4 = \Sigma (x - \bar{x})^4$$

$$m_4 = \frac{s_4}{N}$$

$$m_2 = \frac{s_2}{N}$$

$$\beta_2 = \frac{m_4}{(m_2)^2}$$

```
header = 'Header'
mean = 4.6
```

```
column = ['Header', '2', '6', '3', '8', '4']
mnt2_4 = eu.Statistics.mnt(0, header, mean, column)
print(mnt2_4)
```

```
['Header', 2.40, 0.36, 1.41]
```

Parameters:

header : ***str***

The name of the column.

mean : ***float***

The average of the column.

column : ***str list***

List of values.

Returns:

list

Header of the column as a ***str***.

Standard deviation as a ***float***.

Skewness as a ***float***.

Kurtosis as a ***float***.

exercise_modules.Statistics.covar

Example:

Covariance

$$\sigma_{xy} = \frac{\sum_{i=1}^{N} (x_i - \bar{x})(y_i - \bar{y})}{N}$$

```
mean = ['Header', 4.6]
column = ['Header', '2', '6', '3', '8', '4']
mean_1 = ['Header 1', 7.2]
column_1 = ['Header 1', '9', '5', '9', '6', '7']
col12_covar = eu.Statistics.mnt(0, mean, mean_1, column,
                                column_1)
print(col12_covar)
```

```
-2.92
```

Parameters:

mean : ***list***

The first column's header (***str***) and mean (***float***).

mean_1 : ***list***

The second column's header (***str***) and the mean (***float***).

column : ***str list***

List of values.

column_1 : ***str list***

List of values.

Returns:

float

Covariance between two columns.

exercise_modules.Statistics.cor

Example:

Correlation

$$\rho = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$

```
mean = ['Header' , 4.6]
column = ['Header', '2', '6', '3', '8', '4']
mean_1 = ['Header 1' , 7.2]
column_1 = ['Header 1', '9', '5', '9', '6', '7']
column_mnt2_4 = eu.Statistics.mnt(0, mean[0], mean[1],
                                  column)
column_1_mnt2_4 = eu.Statistics.mnt(0, mean_1[0],
                                       mean_1[1],
                                       column_1)
col12_covar = eu.Statistics.covar(0, mean, mean_1, column,
                                  column_1)
col12_cor = eu.Statistics.cor(0, col12_covar,
                              column_mnt2_4[1],
                              column_1_mnt2_4[1])
print(col12_cor)

-0.84
```

Parameters:

covar : ***float***

Population covariance between two columns.

stnd : ***float***

Column one population standard deviation.

stnd_1 : ***float***

Column two population standard deviation.

Returns:

float

Population correlation between two columns.

exercise_modules.Graph

Example:

```
data = [['Date', '509', '510', '511', '512', '513'],
        ['Header', '1', '3', '4', '3', '5']]
P0_graph = eu.Graph(data, date_col_num, data_col_num)
```

Parameters:

data : ***str list***

Two dimensional list with each column as a list.

date_col_num : ***int***

The list in `data` that has date values.

data_col_num : ***int***

The list in `data` where data columns begin.

Returns:

The graph class object. It only works on a 1-5 scale.

exercise_modules.Graph.hi_lo

Example:

```
4   date_hi_lo = P0_graph.hi_lo(0)
5   print(date_hi_lo)

[513, 509]
```

Parameters:

data_col_num : ***int***

The list in `data` where data columns begin. Built for data on the same scale (i.e. 1-5) or numerical date (MDD i.e. 513 or MMDD i.e. 1002 or YYYYMMDD i.e. 20250513, etc...).

Returns:

list

Highest ***int*** in the column.

Lowest ***int*** in the column.

exercise_modules.Graph.binned

Example:

```
P0_binned = P0_graph.binned(P0_hi_lo)
print(P0_binned)

[[['513', '5']], [['511', '4']], [['510', '3'], ['512', '3']],
  [], [['509', '1']]]
```

Parameters:

hi_lo : ***list***

Contains the highest and lowest value in the column.

Returns:

list

The list contains a sublist for each value sorted from 5 to 1. Each sublist contains the date and value from oldest to newest.

exercise_modules.Graph.time_series

Example:

```
P0_time_series = P0_graph.time_series(date_hi_lo,
                                      P0_binned)
```

Parameters:

hi_lo : ***list***

Contains the highest and lowest value in the column.

binned : ***list***

Output from `exercise_modules.Graph.binned`.

Returns:

list

The first element is a list of lists with spaces ***str*** matching the output from `exercise_modules.Graph.binned`. The second element's an ***int*** scale from 5-1.

exercise_modules.Graph.time_series_print

Example:

```
P0_graph.time_series_print(P0_time_series[0],
                           P0_time_series[1])
```

Parameters:

spaces : ***list***

The list of ***str*** spaces output from `exercise_modules.Graph.time_series` .

scale : ***list***

The ***int*** scale list output from `exercise_modules.Graph.time_series` .

Returns:

graph

```
5         +
4     +
3   +   +
2
1 +
  5 5 5 5 5
  0 1 1 1 1
  9 0 1 2 3
```

Outside of part ABC in exercise_modules.py:

Rudimentary implementation of the merge sort algorithm was used to alphabetize activity for binning and graphing frequency by unique type. Built-in Python methods for ‘replace’, ‘split’, etc are located in ‘202504191600-Cython_clones.py’ and were not used for stability .

https://github.com/DavidLeifer/OpenSourceDocument/blob/main/ExerciseStatisticsVersions/exercise_modules/exercise_module.py

https://github.com/DavidLeifer/OpenSourceDocument/blob/main/ExerciseStatisticsVersions/GraphsExerciseStatistics.ipynb

Statistics:

https://github.com/DavidLeifer/OpenSourceDocument/blob/main/ExerciseStatisticsVersions/GraphExerciseStatisticsNotebooks/202502062012-GraphsExerciseStatistics.ipynb

TODO

TODO 0) the counter variable in mnt was left at sample n-1 instead of n -> update github

TODO 1) change the path from the class to the first two def functions and remove self from all of the functions in the .py file and the 0 self reference in the functions.

2) 202504191600-Cython_clones.py

3)
```
# todo0  rewrite parser for unicode csv str/int.
  # todo1  Stats class avoid NA, NAAN, -9999, etc.
  # todo2  refractor RGB_graphs.
  # todo3  monthly means on bar graphs.
  # todo4  organize merge_sort into another classe.
Complete
  # todo5  modify merge sort to accept entire CSV.
Class
  # todo6  Handle multi word activity descriptions consistently.
Class
  # todo7  switch the second capital letter to lower case if exists.
Class
  # todo8  unchain the four merge sort functions.
Class
```

WorksCited.html

42	Analyzing and Interpreting Data From Likert-Type Scales

23	Kurtosis as Peakedness, 1905-2014 R.I.P.	Maintains that kurtosis is not a measure of peakedness of a distribution but rather a measure of existing outliers in a sample or propensity to produce more outliers in a population.	https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4321753/
		High measures of kurtosis (over 3) have heavy tails or outliers while under 3 would have lighter tails/outliers in the normal distribution.	https://www.itl.nist.gov/div898/handbook/eda/section3/eda35b.htm
		Cronenbach alpha coefficient - measures internal consistency among a survey	https://stats.oarc.ucla.edu/spss/faq/what-does-cronbachs-alpha-mean/
		ANOVA - ???	https://sphweb.bumc.bu.edu/otlt/mph-mod

			ules/bs/bs704_hypothesistesting-anova/bs704_hypothesistesting-anova_print.html
		chi-test - categorical data	https://support.google.com/docs/answer/7004263?hl=en&ref_topic=3105600
24	A Language, not a letter: Learning Statistics in R, Chp 23: What is ARIMA	ARIMA is for data that is stationary or has variance, mean, and autocorrelation that does not change over time. This page provides historical backgrounds of the formula (not in depth).	https://ademos.people.uic.edu/Chapter23.html
25	NIST Engineering Statistics Handbook: 6.4.4.2. Stationarity	Briefly describes what stationarity is with two graphs and one equation.	https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc442.htm
26	Penn State Stats 510: Section 4.1: Seasonal ARIMA models	Provides example calculations for the seasonal ARIMA model. SARIMA is used for data that has seasonal fluctuations and would be most applicable to yearly	https://online.stat.psu.edu/stat510/lesson/4/4.1

		data. Maybe one could	
		apply the techniques to differences in >30 daily values in the same season but with values that differ during the time of day? That last part is speculation I haven't read too deeply.	
27	A Systematic Review of Time Series Classification Techniques Used in Biomedical Applications	Literature review on several hundred papers on deep learning for large, non parametric datasets. A combination of several algorithms performed exceptionally well in prediction tasks	https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9611376/
		but the number of data points are in the several thousands. It is worth further investigation because of Gaussian Process Latent Variable Models which	

		attempt to recognize patterns	
		by shape, thereby reducing dimensional complexity and computation power needed.	
28	Non-parametric inference	Defines non parametric as a group of statistical tests that use data with large and infinitely-dimensional models.	https://statistics.berkeley.edu/research/nonparametric-inference
29	Penn State Stats 462: Applied Regression Analysis. Lesson 10: Autocorrelation	For use in time series data when typical formulas fail to find any trends. The chapter includes many references to formulas and explanations.	https://online.stat.psu.edu/stat462/node/188/
30	UCLA Advanced Research Computing. Statistical Methods and Data Analytics. FAQ: What's with the different formulas for kurtosis?	Provides three formulas for kurtosis calculation.	https://stats.oarc.ucla.edu/other/mult-pkg/faq/general/faq-whats-with-the-different-formulas-for-kurtosis/

31	NIST Engineering Statistics Handbook: 1.3.5.11. Measures of Skewness and Kurtosis	Another referenced source for kurtosis and skewness formulas	https://www.itl.nist.gov/div898/handbook/eda/section3/eda35b.htm
32	International Encyclopedia of Statistical Science. Skewness	Another college reference for skewness.	https://lbj.utexas.edu/sites/default/files/file/news/Skew.pdf